

Lab Assignment 6

6/03/2026

Total Marks: 25

-
- Write the query, execute and paste screenshots, and make a pdf. Paste the text of the query too. You need to show a screenshot to demonstrate the correctness of your query. If you have made some changes that are not visible in the direct output, display the whole table again.
 - Submit one file .pdf file containing all answers and the file name should be **<roll no>_<name>_assignment3.pdf**
 - Write the questions before the answers.
 - Write a small justification for your solution.
 - Complete the exercise before 4.45 PM and submit the pdf in Moodle.
 - You can use PGAdmin for this lab
 - Use the University Database for this lab. You can restore the database from the attached [SQL](#) files
-

1.

A. Create a login role named student_portal_user with password postgres123.

Verify the role creation by listing all existing roles. (1)

B. Grant SELECT privilege on all tables in the public schema to the role

student_portal_user. Verify that the role can retrieve data from tables such as

student, course, and department. (1)

2. Grant INSERT and DELETE privileges on the student table to student_portal_user. Insert a new student record into the student table as student_portal_user. Delete the same record using student_portal_user. (2)

3.

- A. Create a group role named `faculty_group`. Grant UPDATE privilege on the course table to `faculty_group` (1)
- B. Create a login role named `assistant_professor` with password 'prof123' Add `assistant_professor` as a member of `faculty_group` (1)
- C. Verify that the role `assistant_professor` automatically inherits the UPDATE privilege on the course table. (1)

4. Create a role named `academic_admin` with login privilege. Change the session default for the configuration variable `log_statement` to 'all' for the role `academic_admin`. Verify that the change has been applied. (2)

5. Remove the role `student_portal_user` from the PostgreSQL database. Ensure that any dependent privileges are handled appropriately before dropping the role. (2)

6. Create a view named `computer_science_students` that shows only students from the Computer Science department. Use the WITH CHECK OPTION clause to ensure that inserts through the view cannot add students from other departments. Attempt to insert a student from another department and verify that the insertion fails. (2)

7.

- A. Create a view named `dept_summary` showing department-wise counts of students, instructors, courses and average student credits (2)
- B. Insert the following into the student table and check whether they are reflecting in the view or not. Explain why? (1)

Id: 1902, name: Mike, dept_name: statistics, total_credits: 19

8.

- A. Create a new PostgreSQL user analyst_user with password 'analyst123'. Grant them SELECT access only on the view dept_summary, but give them no privileges on the underlying student, instructor, course or department tables. Login as analyst_user and verify whether he is able to access the dept_summary view or not. (1)
- B. Try to directly query the student table as analyst_user. What would happen now? Explain why? (1)
- C. Now try inserting a new row into the dept_summary view. What would happen now? Explain why? (1)

9.

- A. Create a view that shows each classroom's utilization in each semester (consider no of sections in each classroom as classroom's utilization) (2)
- B. Now check how many classrooms were vacant each semester. (1)

10.

- A. Create a view that computes enrollment per section and flags low-enrollment sections (consider a section as low-enrollment if it has less than 300 students enrolled) (2)
- B. Using this view, find the proportion of the number of sections with low enrollment (1)

Note: Do not use CTEs for writing queries.

Schema -

